

Cross-Residual Graph Neural Networks for Biomolecular Graph Classification: An Empirical Study of Residual Routing

Xuelin Hu¹ and Xiaoqin Fu²

¹School of Railway Communication and Signaling Engineering, Liuzhou Railway Vocational Technical College, Liuzhou 545000, China

²School of Railway Locomotive and Rolling Stock, Liuzhou Railway Vocational Technical College, Liuzhou 545000, China

Corresponding author:
Xiaoqin Fu²

Email address: fuxiaoqin@ltzy.edu.cn

ABSTRACT

Background. Graph neural networks are widely used for graph classification on proteins and molecules, but deeper message passing can blur discriminative local structure, and standard skip connections do not clarify whether cross-branch historical exchange offers a better trade-off than ordinary residual reuse. This issue is especially relevant in biomolecular graph classification, where useful evidence may depend on both local motifs and graph-level context.

Methods. We study Cross-Residual Graph Neural Networks (CR-GNN) as a controlled comparison framework that evaluates plain stacking, node-level residual reuse, node-level cross-residual exchange, graph-level residual propagation, and graph-level cross-summary exchange under shared graph-classification backbones. The main benchmark uses six datasets, nine methods, four message-passing operators, and stratified five-fold evaluation with mean best test accuracy and fold-level standard deviation. We further organize targeted experiments on gate control, residual routing modes, residual-parameter sweeps, and exploratory sensitivity scans.

Results. The five-fold results show that residual reuse remains the stronger default family across the biomolecular benchmark package, while cross-residual routing usually improves over plain stacking and yields selective gains on specific datasets rather than a uniform advantage. Within the biological core, GraphResGNN is strongest on PROTEINS and ENZYMES, whereas NodeResGNN is strongest on DD. The targeted routing analyses further show that routing design materially affects performance: learnable dense routing remains strongest on the best graph-level residual line, whereas mild sparse routing is strongest on three of four representative routing targets.

Conclusions. CR-GNN is best interpreted as a controlled biomolecular graph-classification design space rather than a universally dominant architecture. The empirical evidence supports a bounded conclusion: cross-residual interaction is useful but conditional, residual reuse remains the stronger default reference, and routing strategy is an important determinant of performance.

INTRODUCTION

Graph neural networks (GNNs) are now a standard approach for learning from molecular and protein graphs because they encode local neighborhood structure together with higher-order relational context (Kipf and Welling, 2016; Gilmer et al., 2017; Hamilton et al., 2017). In biomolecular graph classification, this combination matters because predictive evidence may depend simultaneously on local biochemical motifs and broader graph-level organization. Benchmark suites such as PROTEINS, DD, ENZYMES, MUTAG, AIDS, and Mutagenicity therefore remain useful as controlled testbeds for methodology-oriented graph classification studies rather than as direct proxies for downstream biological discovery.

The main methodological difficulty is that deeper message passing is not automatically beneficial in these settings. Repeated propagation can suppress informative intermediate states

through over-smoothing and related depth effects (Li et al., 2018; Oono and Suzuki, 2020; Chen et al., 2020). Residual connections, normalization, and regularization alleviate some of this degradation (He et al., 2016; Bresson and Laurent, 2017; Zhao and Akoglu, 2020; Cai et al., 2021; Rong et al., 2019), but most residual designs still reuse information along the same branch. As a result, they improve optimization without directly testing whether alternative information-flow topologies preserve more useful historical signals.

Related graph-learning work has already explored skip connections, dense aggregation, APPNP-style propagation, and Jumping Knowledge readout (Xu et al., 2018; Klicpera et al., 2018; Gao and Ji, 2019). These methods preserve information from multiple depths, yet they do not directly isolate whether exchanging hidden states across branches or reinjecting pooled graph summaries across block boundaries is preferable to ordinary same-path residual reuse. This is the narrower gap addressed in the present paper.

This paper studies Cross-Residual Graph Neural Networks (CR-GNN), a controlled model family that compares plain stacking, node-level residual reuse, node-level cross-residual exchange, graph-level residual propagation, and graph-level cross-summary exchange under shared graph-classification backbones. The central question is deliberately narrower than a generic model race: when does cross-residual interaction help in biomolecular graph classification, when does residual reuse remain the stronger default, and how does residual routing design change that answer? Framing the study in this way is important for the paper’s methodological scope. The goal is not to claim that one additional pathway always yields a better GNN, but to identify which reuse topology remains stable once backbone, data split, and training protocol are controlled.

The work is organized as a biomolecular graph-classification methods study with a mechanism-oriented empirical focus. The main evidence comes from a protein-oriented benchmark core built around PROTEINS, DD, and ENZYMES, together with supplementary molecular robustness datasets. This design keeps the paper close to biologically meaningful graph settings while maintaining a reproducible and controlled benchmark scope. The later sections therefore move from the architectural motivation to the common methodological setting, then to the confirmatory benchmark results, targeted ablations, and supporting sensitivity analyses. This progression is intentional: the benchmark first establishes the empirical pattern, and the gate and routing studies then test whether that pattern can be explained by how residual information is passed forward.

Contributions

The main contributions of this work are:

- Empirical finding. Under controlled five-fold evaluation with shared backbones and protocol, residual reuse (NodeResGNN, GraphResGNN) remains the stronger default family for biomolecular graph classification, while cross-residual routing provides selective rather than uniform gains. This bounded conclusion contrasts with the stronger claim that cross-residual interaction should replace ordinary residual reuse.
- Routing mechanism analysis. Beyond whether reuse exists, how residual information is filtered matters. The comparison of learnable, top-k, and sparse routing modes shows that routing strategy is target-dependent: dense learnable routing remains strongest on the best graph-level residual line, while mild sparse routing is frequently advantageous on cross-oriented settings.
- Location of reuse. The distinction between node-level and graph-level information reuse is practically important. Node-level cross exchange is the more stable cross-oriented design, while graph-level cross routing is more dataset-sensitive and does not consistently match the strongest graph-level residual lines.

MATERIALS AND METHODS

This section defines the common graph-classification setting, the CR-GNN architecture family, the benchmark package, and the shared experimental protocol. The guiding principle is to isolate where information is reused while keeping the remaining pipeline as consistent as possible.

That design choice is central to the paper’s methodological justification: if backbone operator, pooling, and evaluation protocol are held fixed, then performance differences are more plausibly attributable to the reuse topology itself rather than to unrelated implementation changes.

Problem definition and notation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote an undirected or directed graph, where $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$ is the set of n nodes and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the set of edges. Each node $v_i \in \mathcal{V}$ is associated with a feature vector $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$, and the node features for the entire graph are collected in $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$. The graph structure is represented using an adjacency matrix $\mathbf{A}$.

Given a training dataset $\mathcal{D} = \{(\mathcal{G}_i, y_i)\}_{i=1}^N$, the objective is to learn a mapping function $f_{\theta} : \mathcal{G} \rightarrow \hat{y}$ that predicts the label of an unseen graph. The present study compares five architectures under the same graph-classification pipeline:

- PlainGNN: sequential message passing without explicit state reuse
- NodeResGNN: single-branch node-level residual reuse
- NodeCrossGNN: two-branch node-level cross exchange
- GraphResGNN: graph-level residual propagation across sequential blocks
- GraphCrossGNN: graph-level cross-summary exchange across paired blocks

The comparison is designed to isolate where information is reused: inside one branch, across parallel branches, or through pooled graph summaries passed between blocks. This isolates a concrete research question behind the architecture family. Same-branch residual reuse mainly tests whether re-injecting historical states stabilizes deeper propagation, whereas cross-residual designs test whether exchanging information across parallel streams can preserve complementary structure that would otherwise be lost under ordinary stacking.

Shared message passing and graph-level learning

Let Φ denote a message-passing operator. For a chosen backbone, node representations are updated over L layers as

$$\mathbf{h}_i^{(\ell+1)} = \sigma \left(\mathbf{W}^{(\ell)} \cdot \text{AGGREGATE}_{\Phi} \left(\left\{ \mathbf{h}_j^{(\ell)} : j \in \mathcal{N}(i) \right\}, \mathbf{h}_i^{(\ell)} \right) + \mathbf{b}^{(\ell)} \right). \quad (1)$$

After L layers, a readout function R aggregates node-level representations into a graph embedding

$$\mathbf{h}_{\mathcal{G}} = R \left(\left\{ \mathbf{h}_i^{(L)} : i = 1, \dots, n \right\} \right). \quad (2)$$

The classifier produces

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_{\text{clf}} \mathbf{h}_{\mathcal{G}} + \mathbf{b}_{\text{clf}}), \quad (3)$$

and the model is trained end-to-end by minimizing cross-entropy:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}). \quad (4)$$

CR-GNN architecture family

CR-GNN is defined as a compact family of graph classification architectures. All variants share the same input projection, stacked message-passing layers, graph-level pooling, and linear classifier; they differ only in how intermediate node states and graph-level summaries are reused across depth and across branches. The design deliberately avoids introducing separate encoders, task-specific feature engineering, or architecture-specific readout heads, because such additions would make it harder to interpret whether any observed gain comes from residual topology or from extra model capacity.

135 Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a graph with node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ and adjacency structure $\mathbf{A}$.
 136 The model can be instantiated with one of several standard message-passing operators,

$$\Phi \in \{\text{GCN}, \text{GAT}, \text{Graph-Transformer}\},$$

137 and reuses the same operator type throughout a model instance. For a hidden width d , every
 138 branch starts with an input projection

$$\mathbf{H}^{(0)} = \text{ReLU}(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A})), \quad (5)$$

139 followed by L hidden layers and a readout

$$\mathbf{g} = \text{Pool}(\mathbf{H}^{(L)}). \quad (6)$$

140 Single-branch and node-level residual variants

141 PlainGNN. The plain baseline stacks L copies of the same operator without residual reuse:

$$\mathbf{H}^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_{\ell}(\mathbf{H}^{(\ell)}, \mathbf{A})\right)\right). \quad (7)$$

142 NodeResGNN. The first residual variant stores the previous hidden state within the same
 143 branch:

$$\mathbf{H}^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_{\ell}(\mathbf{H}^{(\ell)} + \mathbf{H}^{(\ell-1)}, \mathbf{A})\right)\right). \quad (8)$$

144 NodeCrossGNN. The node-level cross-residual model maintains two parallel branches with
 145 separate input projections:

$$\mathbf{H}_1^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_{\ell}^{(1)}(\mathbf{H}_1^{(\ell)} + \tilde{\mathbf{H}}_2^{(\ell)}, \mathbf{A})\right)\right), \quad (9)$$

$$\mathbf{H}_2^{(\ell+1)} = \psi\left(\text{ReLU}\left(\Phi_{\ell}^{(2)}(\mathbf{H}_2^{(\ell)} + \tilde{\mathbf{H}}_1^{(\ell)}, \mathbf{A})\right)\right). \quad (10)$$

146 The final node representation is the branch sum

$$\mathbf{H}^{(L)} = \mathbf{H}_1^{(L)} + \mathbf{H}_2^{(L)}. \quad (11)$$

147 Graph-level residual propagation

148 GraphResGNN. This variant chains three graph-conditioned blocks:

$$\mathbf{g}^{(t+1)} = B_{t+1}(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(t)}), \quad t \in \{1, 2\}. \quad (12)$$

149 GraphCrossGNN. Four blocks are organized as two sequential pairs. Within each stage, the
 150 two block outputs are swapped and reused as the next stage's graph-level input:

$$(\mathbf{g}_1^{(t)}, \mathbf{g}_2^{(t)}) = (B_{2t-1}(\mathbf{X}, \mathbf{A}, \mathbf{g}_1^{(t-1)}), B_{2t}(\mathbf{X}, \mathbf{A}, \mathbf{g}_2^{(t-1)})), \quad (13)$$

$$(\mathbf{g}_1^{(t+1)}, \mathbf{g}_2^{(t+1)}) = (\mathbf{g}_2^{(t)}, \mathbf{g}_1^{(t)}). \quad (14)$$

151 The final graph embedding is

$$\mathbf{g}_{\text{final}} = \mathbf{g}_1^{(T)} + \mathbf{g}_2^{(T)}. \quad (15)$$

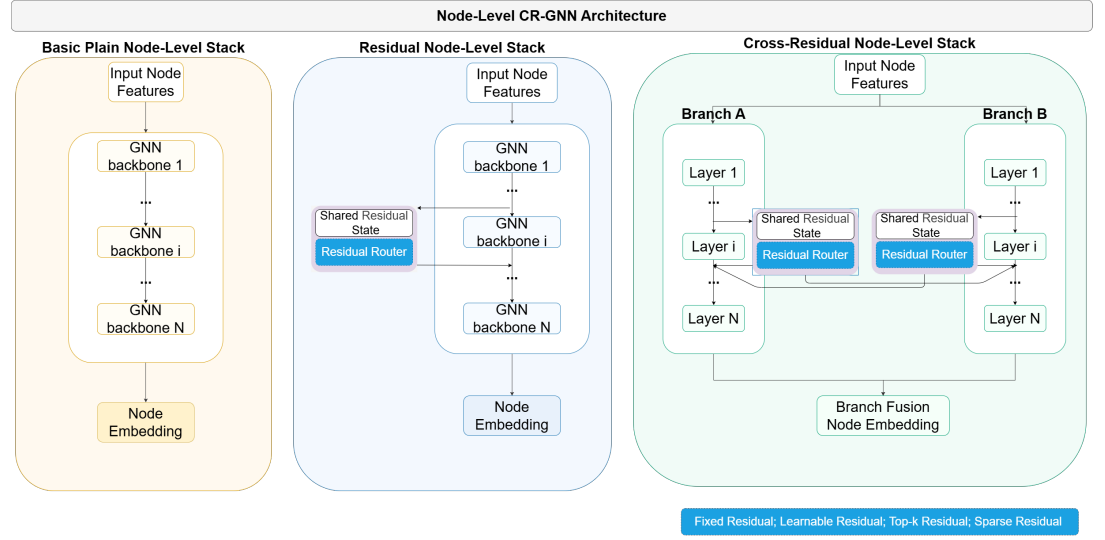


Figure 1. Node-Level CR-GNN Architecture. The three columns compare plain stacking (left), same-branch residual reuse (middle), and two-branch cross-residual exchange (right). In the cross-residual design, each branch reuses not only its own previous state but also the counterpart branch’s state from the previous layer, with residual routers controlling the strength and sparsity of cross-branch injection.

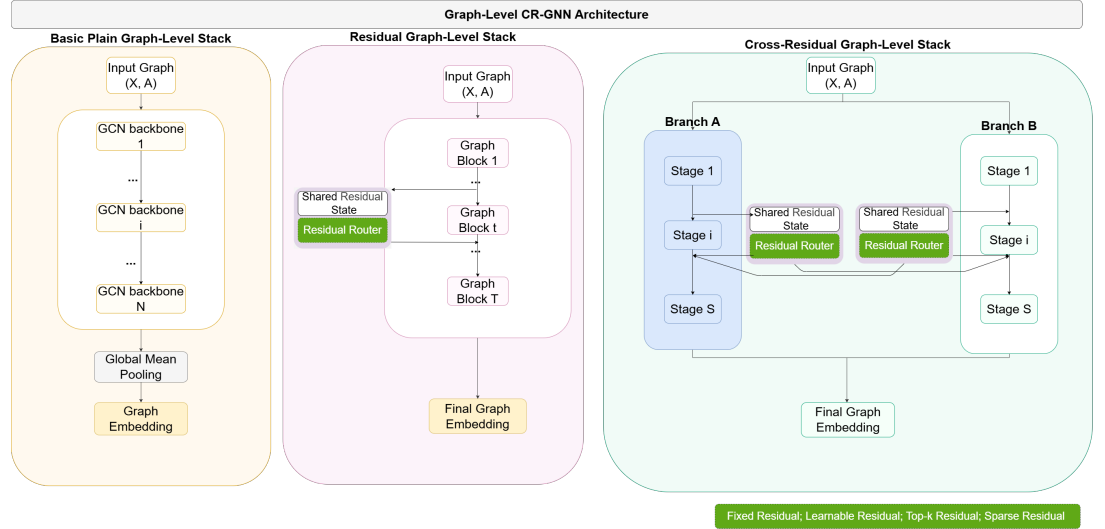


Figure 2. Graph-Level CR-GNN Architecture. The three columns compare plain graph-level stacking (left), sequential graph-level residual propagation (middle), and cross-summary exchange across paired blocks (right). In the cross-residual design, shared residual states and residual routers control graph-level information exchange between parallel block sequences, with four routing modes available: fixed, learnable, top-k, and sparse residual.

152 Residual routing mechanisms

153 Beyond the architecture family itself, this study evaluates how residual information is injected.
154 This second layer is necessary because the architecture comparison alone cannot answer whether
155 an apparent residual advantage comes from the existence of an extra path or from the way that
156 path is filtered. Three routing modes are used in the targeted ablations:

- 157 • Learnable routing: dense residual injection modulated by an adaptive gate.
- 158 • Top-k routing: only the strongest residual channels are retained before injection.
- 159 • Sparse routing: weak residual coefficients are suppressed, leaving a filtered residual path.

160 These routing modes are not used to redefine the core architecture family. Instead, they form a
161 mechanism layer on top of selected residual and cross-residual targets in the experimental design.
162 The routing study is therefore interpretive rather than decorative: it tests whether the same
163 architecture behaves differently when residual information is passed forward densely, selectively,
164 or sparsely.

165 Datasets

166 To test the proposed architecture in a biologically meaningful setting, the benchmarks are orga-
167 nized into a unified two-layer package.

168 The main biological benchmark consists of PROTEINS, DD, and ENZYMES (Fey and
169 Lenssen, 2019; Morris et al., 2020). The supplementary robustness package consists of MU-
170 TAG, AIDS, and Mutagenicity. This organization keeps the main biological claim centered on
171 protein-oriented datasets while still testing structural robustness on additional molecular graphs.

172 PROTEINS PROTEINS contains graph representations of proteins, where nodes denote sec-
173 ondary structure elements and edges encode neighborhood relations between them. The task is
174 binary graph classification. The dataset contains 1,113 graphs, 2 classes, and 3 input features
175 per node.

176 DD DD is a protein structure dataset in which each graph corresponds to a protein and nodes
177 represent amino acids or structure-related units linked by proximity-derived edges. The task is
178 binary classification. DD contains 1,178 graphs, 2 classes, and 89 input features.

179 ENZYMES ENZYMES is a 6-class enzyme classification benchmark with 600 graphs and 3
180 input features. It preserves an enzyme-oriented biological interpretation while remaining directly
181 comparable to the other graph classification benchmarks in the study.

182 All active datasets are used through a unified benchmark interface (Fey and Lenssen, 2019;
183 Morris et al., 2020). We keep preprocessing intentionally light and do not introduce task-specific
184 feature engineering, graph augmentation, or edge redesign in the final protocol.

185 All reported results follow the same stratified two-stage evaluation design:

- 186 • Outer evaluation: stratified 5-fold cross-validation at the graph level
- 187 • Inner validation: a stratified validation subset sampled from the training fold with valida-
188 tion ratio 0.1

189 The primary evaluation metric is graph classification accuracy on the held-out test fold. For
190 fold k with test set $\mathcal{T}_k$, the fold accuracy is

$$\text{Acc}_k = \frac{1}{|\mathcal{T}_k|} \sum_{(\mathcal{G}_i, y_i) \in \mathcal{T}_k} \mathbf{1}[\hat{y}_i = y_i]. \quad (16)$$

191 Across the five folds, we report the mean accuracy

$$\overline{\text{Acc}} = \frac{1}{5} \sum_{k=1}^5 \text{Acc}_k \quad (17)$$

192 and the corresponding standard deviation

$$\sigma_{\text{Acc}} = \sqrt{\frac{1}{5} \sum_{k=1}^5 (\text{Acc}_k - \overline{\text{Acc}})^2}. \quad (18)$$

Table 1. Unified summary of the main biological benchmark package.

Dataset	Source	Task	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
PROTEINS	TU	classification	5-fold CV + val	1113	2	3	39.1 / 72.8	main
DD	TU	classification	5-fold CV + val	1178	2	89	284.3 / 715.7	main
ENZYMES	TU	classification	5-fold CV + val	600	6	3	32.6 / 62.1	main

Table 2. Supplementary robustness datasets retained outside the core biological narrative.

Dataset	Source	Task	Split	#Graphs	#Cls.	Feat.	Avg. N / E	Role
MUTAG	TU	classification	5-fold CV + val	188	2	7	17.9 / 19.8	supp.
AIDS	TU	classification	5-fold CV + val	2000	2	38	15.7 / 16.2	supp.
Mutagenicity	TU	classification	5-fold CV + val	4337	2	14	30.0 / 30.6	supp.

193 Compared methods and backbone operators

194 The full paper-facing benchmark compares nine methods. The CR-GNN family contains five
 195 models: PlainGNN, NodeResGNN, NodeCrossGNN, GraphResGNN, and GraphCrossGNN. Four
 196 external baselines are also included in the benchmark package: GraphSAGEBaseline, GINBase-
 197 line, JKNetBaseline, and APPNPBaseline. These baselines are intended to anchor the con-
 198 trolled comparison against representative graph-classification families with distinct propagation
 199 or aggregation behavior, rather than to claim exhaustive coverage of every recent leaderboard
 200 method.

201 Each method is evaluated under four message-passing operators:

- 202 • GCNConv
- 203 • GATConv
- 204 • GINConv
- 205 • SAGEConv

206 Training configuration and implementation details

207 All reported runs follow the same latest versioned experimental workflow. The recommended
 208 workflow entry is `py/run_final_v3_pipeline.py`, and the associated outputs are stored in `logs/LATEST`,
 209 `records/LATEST`, and `runs/LATEST`. Preprocessing is intentionally light, and the final proto-
 210 col does not add task-specific feature engineering, graph augmentation, or custom edge redesign.
 211 This common workflow is important for reliability: every main benchmark and targeted ablation
 212 result reported in the paper is computed under the same split logic, selection rule, and logging
 213 structure.

214 Experimental design

215 The empirical study is organized to separate confirmatory five-fold evidence from exploratory
 216 supporting analyses. The latest benchmark evaluates six datasets, nine methods, four message-
 217 passing operators, and five outer folds under one unified protocol. This full matrix provides the
 218 main comparison used throughout the paper, and the benchmark tables are placed directly in
 219 the corresponding result subsections so that each empirical claim is supported in place.

220 The benchmark is divided into two layers. The biological core consists of PROTEINS, DD,
 221 and ENZYMES, which anchor the main biomolecular interpretation of the paper. A supple-
 222 mentary robustness package extends the analysis to MUTAG, AIDS, and Mutagenicity. This
 223 split keeps the central claim focused while still testing whether the conclusions remain stable on
 224 additional molecular graph settings.

225 Targeted ablations are organized around three mechanism questions. First, gate-control
 226 ablations test whether adaptive residual injection is stronger or more stable than fixed-strength
 227 reuse on a strong graph-level residual target. Second, cross-gate ablations examine settings
 228 where a cross architecture is already locally competitive, asking whether adaptive gating remains

robust in those cases. Third, residual-routing ablations compare learnable, top-k, and sparse routing modes on representative residual- and cross-oriented targets, so that the study can move beyond architecture labels and evaluate how residual information is injected.

Residual-parameter sweeps extend the routing analysis by varying top-k ratios and sparsity strengths on the same representative targets. Older single-fold sensitivity scans over depth, dropout, and learning rate are retained only as exploratory supporting analyses. In the final interpretation, the five-fold latest-version runs are treated as the confirmatory evidence, while the single-fold scans are used only to describe optimization trends rather than to support the main claim.

Evaluation metrics and statistical reporting

The primary metric is graph classification accuracy on the held-out test fold. Main benchmark and targeted ablation results are reported as five-fold mean accuracy with standard deviation. In the final paper, the five-fold latest-version experiments are treated as the main confirmatory evidence, while older single-fold parameter scans are used only as supporting exploratory analyses.

RESULTS

This section reports the final empirical evidence in four layers: the full benchmark, a mechanism-oriented interpretation of the strongest residual patterns, the targeted gate and residual-routing studies, and the exploratory sensitivity analyses. The evidence hierarchy is explicit. The main benchmark and the targeted ablations use the latest stratified five-fold results and are reported as mean best test accuracy $\pm$ fold-level standard deviation, whereas the older sensitivity scans are retained only as supporting trend analyses. This separation matters because the paper’s main claims are intended to rest on repeated-split evidence rather than on one favorable run.

Overall benchmark performance

The final benchmark evaluates six datasets, nine methods, four operators, and five outer folds under one unified protocol. The main text therefore keeps the benchmark tables next to the specific claims they support, rather than separating the tabular evidence into a standalone appendix. This presentation is deliberate: the paper argues for a bounded empirical conclusion, so the tabular evidence is placed where each claim is made and interpreted.

Table 3. Main biological benchmark results. Numbers are mean best test accuracy $\pm$ standard deviation.

Model	PROTEINS	DD	ENZYMES
PlainGNN	0.7053 $\pm$ 0.0271	0.7165 $\pm$ 0.0179	0.2450 $\pm$ 0.0407
NodeResGNN	0.6945 $\pm$ 0.0509	0.7207 $\pm$ 0.0259	0.2650 $\pm$ 0.0661
NodeCrossGNN	0.6963 $\pm$ 0.0433	0.7198 $\pm$ 0.0178	0.3000 $\pm$ 0.0580
GraphResGNN	0.7223 $\pm$ 0.0367	0.6934 $\pm$ 0.0458	0.3117 $\pm$ 0.0746
GraphCrossGNN	0.6864 $\pm$ 0.0465	0.7097 $\pm$ 0.0309	0.3017 $\pm$ 0.0750

Table 3 anchors the paper’s biological argument. GraphResGNN is best on PROTEINS and ENZYMES, whereas NodeResGNN is best on DD, so the core package already favors residual reuse over plain stacking. The cross-residual variants remain competitive, but on the main biological datasets they mostly narrow the gap instead of overturning the residual baseline.

Table 4. Supplementary robustness benchmark results.

Model	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7236 ± 0.0486	0.8455 ± 0.0257	0.7814 ± 0.0235
NodeResGNN	0.7343 ± 0.0734	0.8690 ± 0.0354	0.7897 ± 0.0139
NodeCrossGNN	0.7397 ± 0.0676	0.8735 ± 0.0412	0.8033 ± 0.0159
GraphResGNN	0.7290 ± 0.0585	0.9160 ± 0.0121	0.8022 ± 0.0220
GraphCrossGNN	0.7129 ± 0.0539	0.8710 ± 0.0412	0.8029 ± 0.0182

Table 4 broadens the scope beyond the protein-oriented core. NodeCrossGNN becomes strongest on MUTAG and Mutagenicity, while GraphResGNN remains best on AIDS. This mixed pattern shows why cross-residual routing is worth keeping in the comparison, but also why the paper avoids claiming that cross routing is universally stronger than residual reuse.

Table 5. Winner summary, optimization depth, and parameter count for each dataset.

Dataset	Winner	Mean Acc	Mean Best Epoch	Params
PROTEINS	GraphResGNN	0.7223	58.8	51208
DD	NodeResGNN	0.7207	25.4	22530
ENZYMES	GraphResGNN	0.3117	91.4	52248
MUTAG	NodeCrossGNN	0.7397	88.2	34434
AIDS	GraphResGNN	0.9160	83.8	57928
Mutagenicity	NodeCrossGNN	0.8033	131.2	35330

Table 5 turns the six benchmark outcomes into a compact cross-dataset summary. The winners change with dataset regime, the best epoch ranges from 25.4 to 131.2, and the largest parameter count does not always produce the best mean accuracy. This supports the paper’s main methodological point that information-flow design matters more than simply selecting the heaviest model.

Taken together, the three tables show a consistent hierarchy. Residual reuse is still the stronger default family in the protein-oriented core, cross-residual routing becomes selectively useful on supplementary molecular datasets, and the winner summary confirms that no single complexity trend explains the results by itself. The combination of mean values and standard deviations is also important here: several gaps are modest rather than dramatic, which is exactly why the paper interprets the results as a controlled pattern of relative stability rather than as evidence of universal domination by one design.

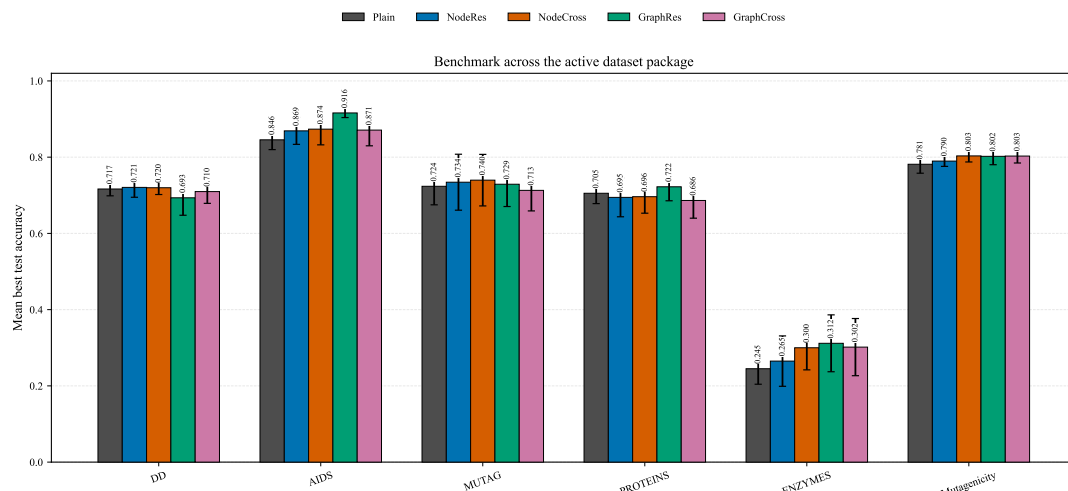


Figure 3. Benchmark over the active dataset package. Each bar reports mean best test accuracy, the numeric label above each bar gives the exact mean value, and error bars show the fold-level standard deviation.

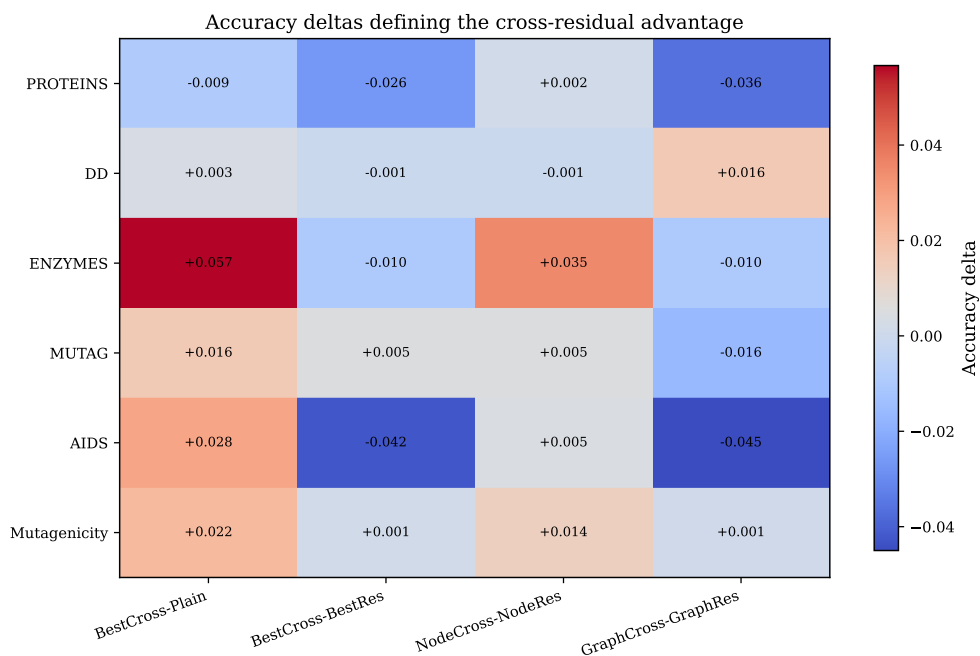


Figure 4. Accuracy deltas that define the practical value of cross-residual routing. Positive values indicate an advantage for the cross design, while negative values indicate that the corresponding plain or residual alternative remains stronger.

277 Figure 3 and Figure 4 make the same point from complementary angles: cross-residual
 278 routing is usually better than plain stacking, but it is not consistently stronger than the best
 279 residual alternative.

Table 6. Main biological benchmark results. Numbers are mean best test accuracy $\pm$ standard deviation.

Model	PROTEINS	DD	ENZYMES
PlainGNN	0.7053 ± 0.0271	0.7165 ± 0.0179	0.2450 ± 0.0407
NodeResGNN	0.6945 ± 0.0509	0.7207 ± 0.0259	0.2650 ± 0.0661
NodeCrossGNN	0.6963 ± 0.0433	0.7198 ± 0.0178	0.3000 ± 0.0580
GraphResGNN	0.7223 ± 0.0367	0.6934 ± 0.0458	0.3117 ± 0.0746
GraphCrossGNN	0.6864 ± 0.0465	0.7097 ± 0.0309	0.3017 ± 0.0750

281 Residual reuse remains the strongest default family in the reported suite. Residual variants win
282 PROTEINS, DD, ENZYMES, and AIDS. In particular, GraphResGNN is best on PROTEINS
283 (0.7223 ± 0.0367), ENZYMES (0.3117 ± 0.0746), and AIDS (0.9160 ± 0.0121), while NodeRes-
284 GNN is best on DD (0.7207 ± 0.0259). The point is not that every residual margin is large, but
285 that residual models repeatedly remain at or near the top across the core biological tasks under
286 the same five-fold protocol.

287 Cross-residual design is selectively strong, not universally dominant. Cross variants still win
288 completed datasets such as MUTAG and Mutagenicity. More importantly, the best cross variant
289 beats PlainGNN on most completed datasets. Therefore, cross routing is clearly more than a
290 design curiosity, but the evidence does not support the stronger claim that cross universally
291 outperforms residual reuse. This narrower reading is important for reliability because it matches
292 both the mean results and the observed split-level variation.

293 Node-level cross is more reliable than graph-level cross. Across the completed datasets, Node-
294 CrossGNN outperforms NodeResGNN more often than GraphCrossGNN outperforms GraphRes-
295 GNN. This suggests that exchanging intermediate node-level states is the more robust cross-
296 residual design in the present study, while graph-level cross routing is more dataset-sensitive.

297 Plain stacking is no longer a credible final baseline. PlainGNN remains competitive on
298 PROTEINS and DD, but it never wins a dataset in the final suite and is usually surpassed by
299 either a residual or a cross-residual variant.

300 Topic-focused results for PROTEINS, DD, and ENZYMES

301 The topic-facing interpretation is anchored in the protein-oriented core datasets. These datasets
302 carry the main substantive weight of the paper, so the interpretation here is intentionally more
303 detailed than in the supplementary package.

304 On PROTEINS, GraphResGNN is clearly strongest, indicating that repeated graph-level
305 residual refinement is the most effective way to preserve useful global structure. The best cross
306 model here is NodeCrossGNN at 0.6963 ± 0.0433 , which improves slightly over NodeResGNN
307 but remains below both PlainGNN and GraphResGNN.

308 On DD, NodeResGNN and NodeCrossGNN are nearly tied, with NodeResGNN retaining
309 a narrow lead. This indicates that node-level hidden-state reuse is important on large sparse
310 protein graphs, but cross exchange does not produce a decisive improvement over simpler residual
311 reuse.

312 On ENZYMES, cross variants become more attractive. GraphCrossGNN reaches $0.3017 \pm$
313 0.0750 and NodeCrossGNN reaches 0.3000 ± 0.0580 , both clearly above PlainGNN at $0.2450 \pm$
314 0.0407 . Even so, GraphResGNN still remains the best model.

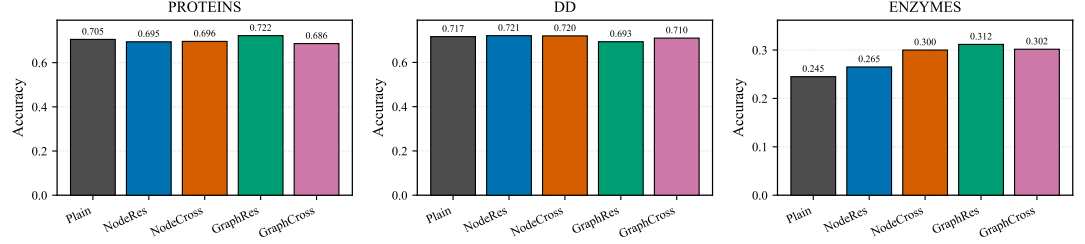


Figure 5. Focused comparison on the completed topic-facing TU datasets. The pattern is mixed rather than one-sided: graph-level residual refinement is strongest on PROTEINS and ENZYMES, while DD favors node-level reuse, with NodeCrossGNN remaining competitive but not dominant.

315 Mechanistic interpretation of the residual advantage

316 The benchmark patterns above are easier to interpret if residual reuse is viewed as a change in
 317 the effective propagation filter rather than only as an optimization trick. Under a linearized plain
 318 update, repeated message passing applies the same propagation operator $\mathbf{S}$ multiple times, so
 319 the hidden state after ℓ layers takes the form $\mathbf{H}^{(\ell)} = \mathbf{S}^\ell \mathbf{X}$. In that setting, deeper propagation in-
 320 creasingly suppresses non-dominant spectral components, which is the standard over-smoothing
 321 tendency. This interpretation does not replace the empirical results, but it provides the missing
 322 intuition for why the residual family may remain stronger even when deeper message passing
 323 alone does not help.

324 Residual reuse changes this behavior. If a residual line reuses the previous hidden state
 325 before the next propagation step, the update no longer corresponds to one pure power of $\mathbf{S}$.
 326 Instead, it produces a polynomial mixture of lower-order and higher-order propagation terms.
 327 In practical terms, the model is no longer forced to rely only on the deepest smoothing order at
 328 a fixed depth; it can preserve more mid-range structural information from earlier layers while
 329 still benefiting from deeper propagation. This interpretation aligns with the empirical fact that
 330 NodeResGNN and GraphResGNN remain stronger default references than PlainGNN across the
 331 biological core.

332 The dataset-specific winner pattern is also consistent with that mechanism view. On PRO-
 333 TEINS and ENZYMES, the strongest model is GraphResGNN, suggesting that re-injecting
 334 graph-level summaries can stabilize useful global context without forcing the model to rely ex-
 335 clusively on deeper propagation powers. On DD, by contrast, NodeResGNN retains the lead,
 336 which indicates that lighter node-level reuse is the more effective way to preserve discriminative
 337 intermediate structure on larger sparse protein graphs. In both cases, the key point is the same:
 338 the best residual models appear to work not simply because they are deeper, but because they
 339 reduce the dominance of the highest propagation order.

340 This mechanism interpretation also clarifies why cross designs are competitive but not uni-
 341 formly dominant. Cross-residual exchange creates additional information routes, yet those extra
 342 routes do not automatically produce a better spectral trade-off than ordinary residual reuse.
 343 The topic-facing results show that NodeCrossGNN is often the more stable cross variant, while
 344 graph-level cross exchange is more regime-sensitive. Therefore, the main comparison should not
 345 be read as “more paths are always better.” A more defensible reading is that the benefit de-
 346 pends on whether the added route preserves informative structure or instead introduces unstable
 347 interactions that the dataset does not reward.

348 The targeted routing experiments sharpen that point further. Once the architecture compari-
 349 son identifies strong residual and cross-oriented targets, the next question is how much historical
 350 information should be passed forward and how selectively that information should be filtered.
 351 The routing results show that dense learnable reuse remains strongest on the best graph-level
 352 residual line, while mild sparse routing is often strongest on representative cross-oriented targets.
 353 This is exactly the kind of behavior expected if residual pathways are useful because they regu-
 354 late the effective propagation mix: some targets benefit from dense preservation of historical
 355 signal, whereas others benefit from suppressing weaker residual channels before they accumulate.

356 The ablations therefore serve a methodological purpose. They show that the reported gains are
 357 not left unexplained after the benchmark stage, but can be probed through controlled changes
 358 to the reuse mechanism itself.

359 The scope of this interpretation should remain disciplined. The argument above is an ex-
 360 planatory mechanism, not a full theorem for the nonlinear trained models in this paper. It
 361 abstracts away activations, optimization dynamics, and operator-specific details. Even so, it
 362 helps connect the benchmark hierarchy to a coherent empirical story: residual reuse remains
 363 the stronger default because it slows representation collapse more reliably than plain stack-
 364 ing, while cross-residual interaction is useful when the additional exchange pathway preserves
 365 task-relevant information rather than merely adding complexity.

366 Results on the supplementary robustness datasets

367 The supplementary package broadens the scope beyond the biological core without changing the
 368 main conclusion. NodeCrossGNN is strongest on MUTAG and Mutagenicity, whereas GraphRes-
 369 GNN remains strongest on AIDS. This pattern preserves the paper’s main boundary claim:
 370 cross-residual interaction has selective practical value, but the residual family remains the more
 371 reliable default reference across the full suite.

Table 7. Supplementary robustness benchmark results.

Model	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7236 ± 0.0486	0.8455 ± 0.0257	0.7814 ± 0.0235
NodeResGNN	0.7343 ± 0.0734	0.8690 ± 0.0354	0.7897 ± 0.0139
NodeCrossGNN	0.7397 ± 0.0676	0.8735 ± 0.0412	0.8033 ± 0.0159
GraphResGNN	0.7290 ± 0.0585	0.9160 ± 0.0121	0.8022 ± 0.0220
GraphCrossGNN	0.7129 ± 0.0539	0.8710 ± 0.0412	0.8029 ± 0.0182

372 Gate ablation results

373 The gate ablations are designed to test whether adaptive control of residual injection is useful
 374 once the main benchmark has identified strong target configurations. On the strongest graph-
 375 level residual line, AIDS + GraphResGNN + GINConv, the comparison among learnable and
 376 fixed gates is the cleanest direct test of adaptive graph-level reuse. Table 8 shows that the
 377 learnable gate is the strongest setting, although the fixed coefficient 0.5 remains very close. The
 378 cross-gate ablation extends the same comparison to six settings where a cross architecture is
 379 already locally favored. In other words, the gate study addresses a question left open by the
 380 benchmark tables: if reuse helps, is the benefit coming from adaptive control or merely from
 381 adding another injection path with any fixed weight?

382 The fold-level table is useful because it shows where that robustness comes from: the learn-
 383 able gate is not just strong in the average, but remains consistently high across the five AIDS
 384 folds without relying on one unusually favorable split. The fuller cross-gate table also makes
 385 the mixed pattern easier to inspect target by target. The intended interpretation is therefore
 386 constrained. The gate study does not need to show that learnable gating is best on every tar-
 387 get. Instead, the relevant question is whether adaptive control is generally more robust than
 388 fixed injection strengths across the selected residual and cross-residual settings. Table 9 and
 389 Table 11 show that learnable gating is best on three of the six cross-favored targets, while fixed
 390 gates remain competitive and can even be superior on the other three targets. This supports a
 391 robustness claim, not a universal-optimum claim.

392 Residual-routing results

393 The residual-routing analysis is the main mechanism extension of the paper. Four representative
 394 targets compare learnable, top-k, and sparse routing modes under the same five-fold protocol.
 395 The clearest overall signal is that there is no universal winner. Table 12 shows that learnable
 396 remains best on the strongest graph-level residual line (AIDS + GraphResGNN + GINConv),
 397 but sparse routing is strongest on three of the four representative targets, with sparse_0p05
 398 emerging as the most consistent sparse setting. The extended routing table keeps the exact

Table 8. AIDS gate ablation on GraphResGNN + GINConv. Numbers are five-fold mean accuracy $\pm$ standard deviation.

Gate setting	Mean accuracy
Learnable	0.9620 $\pm$ 0.0089
Fixed 0.0	0.9150 $\pm$ 0.0138
Fixed 0.5	0.9605 $\pm$ 0.0144
Fixed 1.0	0.9480 $\pm$ 0.0300

Table 9. Cross-gate ablation on six cross-favored targets. Numbers are five-fold mean accuracy $\pm$ standard deviation.

Dataset	Target model/operator	Learnable	Fixed 0.0	Fixed 0.5	Fixed 1.0
AIDS	NodeCrossGNN + GATConv	0.9045 $\pm$ 0.0165	0.9075 $\pm$ 0.0065	0.9195 $\pm$ 0.0114	0.9105 $\pm$ 0.0241
DD	NodeCrossGNN + GATConv	0.7173 $\pm$ 0.0271	0.7053 $\pm$ 0.0361	0.7156 $\pm$ 0.0225	0.7147 $\pm$ 0.0288
DD	NodeCrossGNN + GINConv	0.7071 $\pm$ 0.0306	0.7012 $\pm$ 0.0198	0.7232 $\pm$ 0.0207	0.7113 $\pm$ 0.0277
ENZYMES	NodeCrossGNN + GATConv	0.2800 $\pm$ 0.0629	0.2850 $\pm$ 0.0627	0.2550 $\pm$ 0.0674	0.2650 $\pm$ 0.0690
MUTAG	GraphCrossGNN + GATConv	0.7558 $\pm$ 0.0495	0.7451 $\pm$ 0.0407	0.7450 $\pm$ 0.0416	0.7504 $\pm$ 0.0779
Mutagenicity	NodeCrossGNN + GATConv	0.8114 $\pm$ 0.0110	0.8033 $\pm$ 0.0192	0.8010 $\pm$ 0.0130	0.8036 $\pm$ 0.0078

parameter-level comparison in the main text, so the reader can see directly that the sparse advantage is not a vague family-level trend but a repeated target-level outcome. This substantially reduces the risk of over-interpreting a single average number, because the comparison is repeated across several targets and routing settings rather than inferred from one isolated benchmark cell.

This pattern supports a more precise conclusion than the architecture-only benchmark alone. The important question is not only whether historical information is reused, but also how aggressively that information is filtered before reuse. Table 13 and Table 15 compress the same result by routing family: the sparse family is strongest on three of the four representative targets, while the learnable family remains strongest on the best graph-level residual line.

Residual-parameter sweep results

The additional routing settings `topk_0p75` and `sparse_0p1` refine the previous result rather than changing it. Table 12 shows that the strongest top-k ratio varies by target, which indicates that top-k routing is not governed by one globally optimal retention level. The sparse sweep also shows a clear failure mode: aggressive sparsity can damage otherwise strong residual lines, especially on the strongest graph-level residual target, where `sparse_0p1` drops to 0.8315 ± 0.0549 .

Exploratory sensitivity analysis

Auxiliary sensitivity checks over depth, dropout, and learning rate on PROTEINS, DD, and ENZYMES do not materially change the main conclusion. On PROTEINS and DD, moderate tuning changes affect the gap between the two cross variants, but they do not overturn the broader conclusion that residual baselines remain the stronger default reference. On ENZYMES, the validation signal is visibly noisier, and promising local settings did not reproduce the original full-suite cross results under the final 5-fold average. These checks are reported for transparency, but they should not be read as evidence at the same level as the repeated-split benchmark and targeted ablations.

DISCUSSION

The main empirical message is now more precise than in the earlier draft. Cross-residual routing is usually beneficial relative to plain stacking, but the gain over residual baselines is conditional and dataset-dependent. The present evidence therefore supports CR-GNN as a meaningful biomolecular graph-classification design space rather than as a universally dominant model family. This framing is important for interpreting both the strengths and the limits of the

Table 10. Fold-level AIDS gate ablation on AIDS + GraphResGNN + GINConv.

Gate setting	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4	Mean $\pm$ Std
Learnable	0.9500	0.9700	0.9700	0.9675	0.9525	0.9620 $\pm$ 0.0089
Fixed 0.0	0.9225	0.9025	0.9125	0.9000	0.9375	0.9150 $\pm$ 0.0138
Fixed 0.5	0.9325	0.9625	0.9675	0.9675	0.9725	0.9605 $\pm$ 0.0144
Fixed 1.0	0.8925	0.9450	0.9700	0.9775	0.9550	0.9480 $\pm$ 0.0300

Table 11. Cross-gate ablation on six cross-favored targets. Numbers are five-fold mean accuracy $\pm$ standard deviation.

Dataset	Target model/operator	Learnable	Fixed 0.0	Fixed 0.5	Fixed 1.0
AIDS	NodeCrossGNN + GATConv	0.9045 $\pm$ 0.0165	0.9075 $\pm$ 0.0065	0.9195 $\pm$ 0.0114	0.9105 $\pm$ 0.0241
DD	NodeCrossGNN + GATConv	0.7173 $\pm$ 0.0271	0.7053 $\pm$ 0.0361	0.7156 $\pm$ 0.0225	0.7147 $\pm$ 0.0288
DD	NodeCrossGNN + GINConv	0.7071 $\pm$ 0.0306	0.7012 $\pm$ 0.0198	0.7232 $\pm$ 0.0207	0.7113 $\pm$ 0.0277
ENZYMES	NodeCrossGNN + GATConv	0.2800 $\pm$ 0.0629	0.2850 $\pm$ 0.0627	0.2550 $\pm$ 0.0674	0.2650 $\pm$ 0.0690
MUTAG	GraphCrossGNN + GATConv	0.7558 $\pm$ 0.0495	0.7451 $\pm$ 0.0407	0.7450 $\pm$ 0.0416	0.7504 $\pm$ 0.0779
Mutagenicity	NodeCrossGNN + GATConv	0.8114 $\pm$ 0.0110	0.8033 $\pm$ 0.0192	0.8010 $\pm$ 0.0130	0.8036 $\pm$ 0.0078

paper: the contribution lies in identifying a reproducible hierarchy of reuse mechanisms, not in claiming a new universal best model.

Cross-residual gains are conditional

The benchmark results do not support the strongest possible claim that cross-residual interaction should replace residual reuse as the default choice. Instead, they support a narrower statement: cross-residual design is often better than plain stacking and can be distinctly useful on selected datasets, but ordinary residual reuse remains the stronger default family across the full biomolecular package. This narrower conclusion is more convincing because it is aligned with the five-fold mean $\pm$ standard deviation reporting rather than with isolated best-case gains.

Node-level and graph-level reuse behave differently

The distinction between node-level and graph-level information reuse is also practically important. Node-level cross exchange is the more stable cross-oriented design, whereas graph-level cross routing is more sensitive to dataset regime and does not consistently match the strongest graph-level residual lines. This suggests that the location of reuse is at least as important as the existence of an additional reuse path.

Residual routing is part of the mechanism story

The residual-routing ablations add an important qualification to the main architecture comparison. The question is not only whether residual information should be reused, but also whether it should be reused densely, sparsified, or filtered by top-k selection. The current results show that routing strategy is target-dependent: learnable dense routing remains strongest on the best graph-level residual line, while mild sparse routing is frequently advantageous on cross-oriented settings. This shifts the paper away from a simple architecture ranking exercise and toward a mechanism study in which architecture and routing are interpreted together.

Evidence hierarchy and study limitations

The strongest evidence in the paper comes from the five-fold LATEST benchmark and the five-fold targeted ablations. By contrast, the older single-fold sensitivity scans are useful only for optimization interpretation. They should not be treated as the same class of quantitative evidence. A second limitation is baseline scope. The benchmark includes representative residual, propagation, and aggregation baselines, but it is not intended as an exhaustive leaderboard against every recent graph-classification method. The paper is instead designed to support a controlled comparison of reuse topologies under a stable protocol. A third limitation is dataset scale and task scope: although the datasets are biomolecular or molecular and the biological

Table 12. Residual-routing and parameter-sweep summary on four representative targets. Numbers are five-fold mean accuracy $\pm$ standard deviation.

Target	Learnable	Top-k 0.25	Top-k 0.5	Top-k 0.75	Sparse 0.02	Sparse 0.05	Sparse 0.1	Best
AIDS + GraphResGNN + GINConv	0.9500 $\pm$ 0.0087	0.9075 $\pm$ 0.0237	0.9330 $\pm$ 0.0251	0.9240 $\pm$ 0.0287	0.9470 $\pm$ 0.0181	0.9250 $\pm$ 0.0152	0.8315 $\pm$ 0.0549	Learnable
PROTEINS + GraphResGNN + SAGEConv	0.7178 $\pm$ 0.0411	0.7241 $\pm$ 0.0407	0.7151 $\pm$ 0.0397	0.7214 $\pm$ 0.0318	0.7160 $\pm$ 0.0248	0.7259 $\pm$ 0.0475	0.7088 $\pm$ 0.0521	Sparse 0.05
AIDS + NodeCrossGNN + GATConv	0.8970 $\pm$ 0.0491	0.9160 $\pm$ 0.0128	0.8960 $\pm$ 0.0483	0.8960 $\pm$ 0.0489	0.9060 $\pm$ 0.0268	0.9180 $\pm$ 0.0162	0.9025 $\pm$ 0.0139	Sparse 0.05
DD + NodeCrossGNN + GATConv	0.7080 $\pm$ 0.0172	0.7062 $\pm$ 0.0249	0.7029 $\pm$ 0.0160	0.7164 $\pm$ 0.0248	0.7147 $\pm$ 0.0259	0.7181 $\pm$ 0.0196	0.7105 $\pm$ 0.0243	Sparse 0.05

Table 13. Family-wise routing winners.

Target	Best learnable	Best top-k	Best sparse	Overall winner
AIDS + GraphResGNN + GINConv	0.9500	0.9330	0.9470	Learnable
PROTEINS + GraphResGNN + SAGEConv	0.7178	0.7241	0.7259	Sparse
AIDS + NodeCrossGNN + GATConv	0.8970	0.9160	0.9180	Sparse
DD + NodeCrossGNN + GATConv	0.7080	0.7164	0.7181	Sparse

core is meaningful, they are still medium-scale benchmark datasets for graph classification. The study therefore supports conclusions about controlled graph-classification behavior, not about large-scale scalability, direct biological discovery, omics integration, or task-specific biological interpretation.

Future directions

The next research step is to test the same information-flow ideas with richer biological inputs, such as sequence-derived embeddings, ontology annotations, or expression-related signals. Another important extension is to evaluate the same controlled reuse questions on larger graph benchmarks, so that the current mechanism-oriented conclusions can be tested beyond the present medium-scale setting. In parallel, the residual-routing perspective introduced here could be extended to broader biomolecular graph tasks where graph topology must be combined with stronger domain supervision rather than benchmark-only evaluation.

CONCLUSIONS

This paper studies CR-GNN as a controlled biomolecular graph-classification framework rather than as a claim of universal model superiority. The final evidence supports a narrower but stronger conclusion: cross-residual interaction is useful, but residual variants remain the strongest default family in the reported benchmark suite.

The main contribution is therefore not a universally dominant architecture, but a controlled comparison of information-flow mechanisms and residual-routing strategies. Across the biological core, GraphResGNN is strongest on PROTEINS and ENZYMES, whereas NodeResGNN is strongest on DD. Cross-residual variants still provide useful gains over plain stacking on selected datasets, but they do not replace residual baselines as the main point of reference. In this sense, the paper’s central finding is comparative and mechanism-oriented: once the training protocol and backbone family are held stable, where residual information is reused matters, and the strongest default remains ordinary residual reuse rather than unrestricted cross exchange.

The routing analysis sharpens that result further. Learnable dense routing remains strongest on the best graph-level residual line, while mild sparse routing is frequently effective on cross-oriented settings. This shows that the practical question is not only whether an additional residual path exists, but how historical information is filtered before reuse. The targeted routing results therefore help explain why the benchmark hierarchy is mixed rather than one-sided.

The scope of this conclusion should remain disciplined. The strongest claims in the paper are supported by the stratified five-fold benchmark and the targeted five-fold ablations, whereas the older sensitivity scans are only supportive. Likewise, the present study addresses controlled graph-classification behavior on medium-scale biomolecular and molecular benchmarks; it does not by itself establish large-scale scalability or direct biological interpretability.

Taken together, these results position CR-GNN as a practical biomolecular graph-classification design space and provide a methodological basis for future work on richer biological inputs, larger

Table 14. Residual-routing and parameter-sweep summary on four representative targets. Numbers are five-fold mean accuracy $\pm$ standard deviation.

Target	Learnable	Top-k 0.25	Top-k 0.5	Top-k 0.75	Sparse 0.02	Sparse 0.05	Sparse 0.1	Best
AIDS + GraphResGNN + GINConv	0.9500 $\pm$ 0.0087	0.9075 $\pm$ 0.0237	0.9330 $\pm$ 0.0251	0.9240 $\pm$ 0.0287	0.9470 $\pm$ 0.0181	0.9250 $\pm$ 0.0152	0.8315 $\pm$ 0.0549	Learnable
PROTEINS + GraphResGNN + SAGEConv	0.7178 $\pm$ 0.0411	0.7241 $\pm$ 0.0407	0.7151 $\pm$ 0.0397	0.7214 $\pm$ 0.0318	0.7160 $\pm$ 0.0248	0.7259 $\pm$ 0.0475	0.7088 $\pm$ 0.0521	Sparse 0.05
AIDS + NodeCrossGNN + GATConv	0.8970 $\pm$ 0.0491	0.9160 $\pm$ 0.0128	0.8960 $\pm$ 0.0483	0.8960 $\pm$ 0.0489	0.9060 $\pm$ 0.0268	0.9180 $\pm$ 0.0162	0.9025 $\pm$ 0.0139	Sparse 0.05
DD + NodeCrossGNN + GATConv	0.7080 $\pm$ 0.0172	0.7062 $\pm$ 0.0249	0.7029 $\pm$ 0.0160	0.7164 $\pm$ 0.0248	0.7147 $\pm$ 0.0259	0.7181 $\pm$ 0.0196	0.7105 $\pm$ 0.0243	Sparse 0.05

Table 15. Family-wise routing winners on representative targets.

Target	Best learnable	Best top-k	Best sparse	Overall winner
AIDS + GraphResGNN + GINConv	0.9500	0.9330	0.9470	Learnable
PROTEINS + GraphResGNN + SAGEConv	0.7178	0.7241	0.7259	Sparse
AIDS + NodeCrossGNN + GATConv	0.8970	0.9160	0.9180	Sparse
DD + NodeCrossGNN + GATConv	0.7080	0.7164	0.7181	Sparse

500 benchmarks, and graph-learning settings where residual reuse must be coordinated with stronger
501 domain supervision.

502 ACKNOWLEDGMENTS

503 The authors thank the open-source graph learning community and the maintainers of the bench-
504 mark resources used in this study. They also thank the reviewers and editors for their time and
505 feedback.

506 REFERENCES

- 507 Xavier Bresson and Thomas Laurent. Residual gated graph convnets. arXiv preprint
508 arXiv:1711.07553, 2017.
- 509 Tianle Cai, Jiacheng Luo, Sheng Wang, Kai Zhang, Yu Tian, Liwei Yang, Zekun Li, and Kilian
510 Weinberger. Graphnorm: A principled approach to accelerating graph neural network training.
511 In International Conference on Machine Learning, pages 1277–1287, 2021.
- 512 Yuning Chen, Xihao Wei, Pan Xu, Xinwen Wang, Ping Luo, Zhiyuan Li, and Jian Tang. Revis-
513 iting over-smoothing in deep gcns. arXiv preprint arXiv:2003.13663, 2020.
- 514 Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric.
515 arXiv preprint arXiv:1903.02428, 2019.
- 516 Hongyang Gao and Shuiwang Ji. Graph u-net. arXiv preprint arXiv:1905.05178, 2019.
- 517 Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural
518 message passing for quantum chemistry. In International Conference on Machine Learning,
519 pages 1263–1272, 2017.
- 520 William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large
521 graphs. In Advances in neural information processing systems, pages 1024–1034, 2017.
- 522 Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image
523 recognition. In Proceedings of the IEEE conference on computer vision and pattern recogni-
524 tion, pages 770–778, 2016.
- 525 Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional net-
526 works. arXiv preprint arXiv:1609.02907, 2016.
- 527 Johannes Klicpera, Alan Bojchevski, and Stephan Günnemann. Combining label propagation
528 and simple models out-performs graph neural networks. arXiv preprint arXiv:1810.05997,
529 2018.

530 Guohao Li, Elias Muller, Abbas Thabet, and Bernard Ghanem. Deeper insights into graph
531 convolutional networks for semi-supervised learning. arXiv preprint arXiv:1809.02584, 2018.

532 Christopher Morris, Roger Wattenhofer, and Kristian Kersting. TUDataset: A collection of
533 benchmark datasets for learning with graphs. arXiv preprint arXiv:2007.08663, 2020.

534 Kenta Oono and Taiji Suzuki. Simple and deep graph convolutional networks. arXiv preprint
535 arXiv:2006.13604, 2020.

536 Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. Dropedge: Towards deep graph
537 convolutional networks on node classification. arXiv preprint arXiv:1907.10903, 2019.

538 Keyulu Xu, Chengtao Li, Yonglong Tian, Xiao Du, Jure Leskovec, and Stefanie Jegelka. Repre-
539 sentation learning on graphs with jumping knowledge networks. In International Conference
540 on Machine Learning, pages 5449–5458, 2018.

541 Lingxiao Zhao and Leman Akoglu. Pairnorm: Tackling over-smoothing in gns. arXiv preprint
542 arXiv:2009.10698, 2020.